

State Machine Generator

User Manual

System Documentation

July 1, 2026

Contents

1	Overview	3
2	Obtaining and Installing	3
3	Generating the Code	3
3.1	Command Line Options	3
3.2	Outputs	4
4	SMB File Structure	4
4.1	Language-Specific Syntax	5
5	Integration & API	5
5.1	Rust Integration	5
5.2	C Integration	6
5.3	Python Integration	6
5.4	TypeScript Integration	7
5.5	Context Struct	8
6	Writing Logic: Handlers vs. Hooks	8
6.1	Local Handlers (Per State/Transition)	8
6.2	Global Hooks (System-wide)	9
6.3	Available Variables in Code Blocks	9
7	Transitions	9
7.1	Basic Transition Syntax	9
7.2	Transition Execution Order	10
7.3	Using Actions (The Mealy Aspect)	10
7.4	Path Syntax	10
7.5	Transition Scenarios	11
7.6	Termination	11
8	History	11
9	Orthogonal States (Parallelism)	12
9.1	Safety: <code>transition_fired</code>	12
9.2	Fork Transitions (Drilling Down)	12
9.3	Cross-Limb Transitions	12
9.4	Joining (Synchronization)	12

10 Decisions (Logic Trees)	13
10.1 Legacy Mode (before <code>SM-builder-version: "0.6.0"</code>)	13
10.2 Hierarchical Mode (<code>SM-builder-version: "0.6.0"</code> and later)	13
10.3 Defining Decisions	14
10.4 Using Decisions	14
11 AND Join Nodes	14
11.1 Defining AND Nodes	15
11.2 Use Case 1 — Synchronization Join (Sibling Regions)	15
11.3 Use Case 2 — Cross-Boundary Join (Any Combination of Sources)	16
11.4 Use Case 3 — Replacing Chained Decisions	17
11.5 Per-Source Guards and the <code>time</code> Variable	17
11.6 AND Node Rules	18
12 Complete Example	18

1 Overview

This tool generates a safety-critical Hierarchical State Machine (HSM) from a YAML/SMB definition file. It supports four output languages — **Rust**, **C**, **Python**, and **TypeScript** — and includes advanced features such as orthogonal regions (parallelism), history patterns, decision trees, and fork transitions.

The generator also produces a **Graphviz DOT** file for visualizing the state machine diagram.

2 Obtaining and Installing

The source code is hosted on GitHub:

<https://github.com/ingvast/SM-code-generator>

Clone the repository and follow the build and installation instructions in the `README.md` file in the repository root. In short, the tool is packaged as a Python wheel and installed via `uv`:

```
uv build
uv tool install dist/smbuilder-*.whl
```

After installation the `sm-compiler` command is available on your `PATH`.

3 Generating the Code

The `sm-compiler` command validates your model and produces code in the selected target language.

```
# Language is read from the 'language:' key in the .smb file
sm-compiler model.smb

# Override the language
sm-compiler model.smb --lang rust
sm-compiler model.smb --lang c
sm-compiler model.smb --lang typescript

# Custom output base path (extensions added automatically)
# Produces build/myfsm.rs and build/myfsm.dot
sm-compiler model.smb -o build/myfsm
```

Listing 1: Command Line Usage

3.1 Command Line Options

Option	Default	Description
file	(required)	Path to the input <code>.smb</code> file.
--lang	from file	Output language: <code>rust</code> , <code>c</code> , <code>python</code> , or <code>typescript</code> . If omitted, reads the <code>language:</code> key from the SMB file (defaults to <code>rust</code> if absent). The <code>language:</code> key may be a list, in which case code is generated for all listed languages.

Option	Default	Description
-o, --output	./statemachine	Output base path without extension. Language-specific extensions (.rs, .c, .h, .py, .ts, .dot) are appended automatically. Parent directories are created if needed.

3.2 Outputs

Given -o path/to/name, the following files are produced:

Language	Output Files	Description
Rust	name.rs	Complete Rust implementation (single file).
C	name.h, name.c	Header with struct/function declarations and source with implementation. The .c file includes name.h automatically.
Python	name.py	Complete Python implementation (single file). Import with <code>from name import StateMachine</code> .
TypeScript	name.ts	Complete TypeScript implementation (single file). Import with <code>import { StateMachine } from "./name"</code> .
All	name.dot	Graphviz visualization of the state machine.

4 SMB File Structure

The input file uses YAML syntax with the extension .smb. A complete model has this top-level structure:

```

language: rust                # Target language (rust, c, python, or
                             typescript)

includes: |                   # Code placed before the Context struct
  // imports, helper functions, etc.

context: |                    # User-defined fields for the Context struct
  pub counter: i32,           # (Rust syntax)
  pub flag: bool,

context_init: |               # Initialization code for context fields
  counter: 0,                  # (Rust: struct init syntax)
  flag: true,

hooks:                        # Global code injected into all states
  entry: |
    // runs on every state entry
  exit: |
    // runs on every state exit
  do: |
    // runs on every state tick
  transition: |
    // runs on every transition

initial: first_state          # Name of the initial child state

```

```

states:
  first_state:
    # ... state definition ...

```

Listing 2: Top-level SMB structure

4.1 Language-Specific Syntax

Since `includes`, `context`, `context_init`, `guards`, and `actions` contain raw target-language code, they must use the syntax of the selected output language:

Element	Rust	C	Python	TypeScript
Context access	<code>ctx.field</code>	<code>ctx->field</code>	<code>ctx.field</code>	<code>ctx.field</code>
Context fields	<code>pub field: Type,</code>	<code>Type field;</code>	<code>field: Type</code>	<code>field: Type = val;</code>
Context init	<code>field: value,</code>	<code>sm->ctx.field = val;</code>	<code>ctx.field = val</code>	<code>ctx.field = val;</code>
Print	<code>println!("{}", ...)</code>	<code>printf("...\n")</code>	<code>print("...")</code>	<code>console.log("...")</code>
String type	<code>&str</code>	<code>const char*</code>	<code>str</code>	<code>string</code>
Boolean	<code>bool</code>	<code>bool</code>	<code>bool</code>	<code>boolean</code>
True/False	<code>true/false</code>	<code>true/false</code>	<code>True/False</code>	<code>true/false</code>
Logical OR	<code> </code>	<code> </code>	<code>or</code>	<code> </code>
Logical AND	<code>&&</code>	<code>&&</code>	<code>and</code>	<code>&&</code>
Equality	<code>==</code>	<code>==</code>	<code>==</code>	<code>===</code>

5 Integration & API

5.1 Rust Integration

The generated code provides a `StateMachine` struct. You must initialize it and call `tick()` cyclically.

```

mod statemachine;
use statemachine::StateMachine;

fn main() {
    let mut sm = StateMachine::new();

    while sm.is_running() {
        sm.ctx.now += 0.1; // Advance logical time
        sm.tick();
        println!("State: {}", sm.get_state_str());
        std::thread::sleep(std::time::Duration::from_millis(100));
    }
}

```

Listing 3: Example Rust driver (main.rs)

Rust API

Function	Location	Description
<code>StateMachine::new()</code>	constructor	Creates and starts the state machine.
<code>sm.tick()</code>	sm	Executes one cycle of logic (do-activities & transitions).
<code>sm.is_running()</code>	sm	Returns <code>false</code> if the machine has terminated.

Function	Location	Description
<code>sm.get_state_str()</code>	<code>sm</code>	Returns a string like <code>/run/active</code> or <code>/p/[a/x,b/y]</code> .
<code>ctx.in_state_X()</code>	<code>ctx</code>	Returns <code>true</code> if state <code>X</code> is currently active.

5.2 C Integration

The generated code provides a `StateMachine` struct with an `SM_Context` inside. Use the provided API functions.

Listing 4: Example C driver (main.c)

```
#include "statemachine.h"
#include <stdio.h>
#include <unistd.h>

int main(void) {
    StateMachine sm;
    sm_init(&sm);

    char state_buf[256];

    while (sm_is_running(&sm)) {
        sm_tick(&sm);
        sm_get_state_str(&sm, state_buf, sizeof(state_buf));
        printf("State: %s\n", state_buf);
        usleep(100000);
    }

    return 0;
}
```

Compile with:

```
gcc main.c statemachine.c -o my_program
```

C API

Function	Description
<code>sm_init(StateMachine* sm)</code>	Initializes context (zeroes memory, applies initial state).
<code>sm_tick(StateMachine* sm)</code>	Executes one cycle of logic.
<code>sm_is_running(StateMachine* sm)</code>	Returns <code>true</code> if the machine has not terminated.
<code>sm_get_state_str(StateMachine* sm, char* buf, size_t max)</code>	Writes the current state path into <code>buf</code> .
<code>IN_STATE_X</code>	Macro: evaluates to <code>true</code> if state <code>X</code> is currently active inside guards and actions where <code>ctx</code> is in scope.

5.3 Python Integration

The generated code provides a `StateMachine` class. No compilation needed — run directly with Python 3.

Listing 5: Example Python driver

```
from statemachine import StateMachine

sm = StateMachine()
print(sm.get_state_str())

while sm.is_running():
    sm.tick()
    sm.ctx.counter += 1
    print(f"{sm.ctx.counter:02d}:{sm.get_state_str()}")
```

Run with:

```
python3 driver.py
```

Python API

Method / Attribute	Description
<code>StateMachine()</code>	Creates and initializes the state machine. Enters the initial state.
<code>sm.tick()</code>	Executes one cycle of logic.
<code>sm.is_running()</code>	Returns <code>True</code> if the machine has not terminated.
<code>sm.get_state_str()</code>	Returns the current state path as a string.
<code>sm.ctx</code>	Direct access to the <code>Context</code> object (read/write fields).
<code>ctx.in_state_X()</code>	Returns <code>True</code> if state <code>X</code> is currently active.

Note: Python uses `None` for empty function pointers and first-class functions for state call-backs. The `context_init` code uses `ctx.field = value` syntax, where `ctx` is an alias for `self` inside the `Context.__init__` method.

5.4 TypeScript Integration

The generated code provides a `StateMachine` class exported from a single `.ts` file. No compilation step is needed — run with `npm run tsx` or import as a module.

```
import { StateMachine } from "./statemachine";

const sm = new StateMachine();
console.log(sm.getStateStr());

while (sm.isRunning()) {
    sm.tick();
    sm.ctx.counter += 1;
    console.log(`${String(sm.ctx.counter).padStart(2, '0')}:${sm.getStateStr()}`);
}
```

Listing 6: Example TypeScript driver

Run with:

```
npm run tsx driver.ts
```

TypeScript API

Method / Attribute	Description
<code>new StateMachine()</code>	Creates and initializes the state machine. Enters the initial state.
<code>sm.tick()</code>	Executes one cycle of logic.
<code>sm.isRunning()</code>	Returns <code>true</code> if the machine has not terminated.
<code>sm.getStateStr()</code>	Returns the current state path as a string.
<code>sm.ctx</code>	Direct access to the <code>Context</code> object (read/write fields).
<code>ctx.inState_X()</code>	Returns <code>true</code> if state <code>X</code> is currently active.

Note: TypeScript uses `null` for empty function pointers and first-class functions for state callbacks. Function pointer fields are typed as `StateFn | null`. The `context_init` code uses `ctx.field = value`; syntax, where `ctx` is an alias for `this` inside the `Context` constructor. Use `===` for equality checks in guards.

5.5 Context Struct

All backends generate a `Context` struct or class (Rust: `Context`, C: `SM_Context`, Python/TypeScript: `Context`) containing:

- `now`: Logical time (`f64/double`). You update this before each tick.
- `state_timers[]`: Per-state entry timestamps (set automatically on entry).
- `transition_fired`: Safety flag for orthogonal regions (managed automatically).
- `terminated`: Set when a `null` transition fires (managed automatically).
- Hierarchy pointers: Function pointers tracking the active state at each level (managed automatically).
- User fields: Whatever you declare in the `context:` block.

6 Writing Logic: Handlers vs. Hooks

6.1 Local Handlers (Per State/Transition)

These are defined inside specific states or transitions in the YAML. They apply only to that specific element.

- `entry`: Runs once when entering the specific state.
- `do`: Runs every tick while in the specific state.
- `exit`: Runs once when leaving the specific state.
- `action`: Runs during a specific transition (see Section 7).

6.2 Global Hooks (System-wide)

Hooks are defined at the root level under the `hooks:` key. They inject code into **every** occurrence of an event type across the entire machine (e.g., for logging).

Hook Key	Trigger Event
<code>hooks: entry</code>	Injected into all states' entry handlers.
<code>hooks: do</code>	Injected into all states' do handlers.
<code>hooks: exit</code>	Injected into all states' exit handlers.
<code>hooks: transition</code>	Injected into all transitions (equivalent to a global action).

Note: There is no `hooks: action`. Use `hooks: transition` instead.

6.3 Available Variables in Code Blocks

The following variables are injected into the scope of your code snippets. Use the appropriate syntax for your target language.

Variable	Scope	Description
<code>ctx</code>	All	The shared context struct. Rust: <code>&mut Context</code> . C: <code>SM_Context*</code> . Python/TypeScript: <code>Context</code> instance.
<code>time</code>	All	Time elapsed (seconds) since entering the current state (<code>f64/double</code>).
<code>state_name</code>	All	Short name of the current state (e.g., <code>"idle"</code>).
<code>state_full_name</code>	All	Full path of the current state (e.g., <code>"/run/idle"</code>).
<code>t_src</code>	Transitions	The full path of the source state.
<code>t_dst</code>	Transitions	The full path/description of the target.

7 Transitions

A transition determines how the machine moves between states.

7.1 Basic Transition Syntax

```
transitions:
- guard: ctx.counter == 5      # Condition (target language expression)
  action: |                    # Code executed during the transition
    // runs BEFORE exit/entry sequence
  to: target_state             # Where to go
```

Listing 7: Transition syntax

All three fields are optional:

- Omitting `guard` makes the transition unconditional (`true`).
- Omitting `action` means no transition-time code.
- `to: null` triggers machine termination.

7.2 Transition Execution Order

When a transition fires:

1. Evaluate the `guard` condition.
2. Set `transition_fired = true`.
3. Execute `action` code (if any).
4. Execute the exit sequence (from current state up to the Least Common Ancestor, bottom-up).
5. Execute the entry sequence (from LCA down to target state, top-down).

7.3 Using Actions (The Mealy Aspect)

The `action` keyword allows you to execute code *during* the transition, before the old state is exited.

```
states:
  reading:
    transitions:
      - guard: ctx.value > 100
        action: |
          println!("Moving_{}_->_{}", t_src, t_dst);
          ctx.log_alert("Threshold_Exceeded");
        to: alert_mode
```

Listing 8: Rust example using action variables

```
states:
  reading:
    transitions:
      - guard: ctx->value > 100
        action: |
          printf("Moving_%s->%s\n", t_src, t_dst);
          log_alert(ctx, "Threshold_Exceeded");
        to: alert_mode
```

Listing 9: C example using action variables

7.4 Path Syntax

The `to:` keyword supports several addressing modes:

Syntax	Description
/absolute/path	From the root. Use for long jumps across the hierarchy.
sibling	A state in the same parent (lateral transition).
./child	A direct child of the current state (drill down).
../sibling	Move up one level to the parent and find a sibling (a child of the parent). Equivalent to bare <code>sibling</code> . Each additional <code>../</code> climbs one more level, so <code>../../uncle</code> is a sibling of the parent, <code>../../../x</code> a sibling of the grandparent, and so on.
.	Self-transition: the state exits and immediately re-enters.
null	Machine termination. Exits all states up to root and stops.

Syntax	Description
@Name	Delegate to a named decision or AND pseudo-state (see Sections 10–11). The path prefix follows exactly the same rules as for state paths: bare @Name resolves in the calling state’s parent container, ./Child/@Name goes into a child, ../Sibling/@Name steps up then down, /@Name is an absolute root reference, etc.
/path/[a/tgt, b/tgt]	Explicit fork into an orthogonal state (see Section 9).

7.5 Transition Scenarios

Lateral (Sibling to Sibling)

- **Exits:** A
- **Enters:** B

Drill Down (Parent to Child)

- **Exits:** None (P remains active).
- **Enters:** C

Move Up (Child to Parent/Uncle)

- **Exits:** C, then Parent P.
- **Enters:** U

7.6 Termination

Setting `to: null` causes the machine to exit all states from the current state up through root, then stop. After termination, `is_running()` (Rust/Python), `isRunning()` (TypeScript), or `sm_is_running()` (C) returns false.

8 History

A composite state with `history: true` remembers its last active child. When re-entered, it resumes from that child instead of the `initial` state.

```
process:
  history: true
  initial: step_a
  states:
    step_a:
      transitions:
        - guard: ctx.counter == 3
          to: step_b
    step_b:
      transitions:
        - guard: ctx.counter == 5
          to: /other_state
```

Listing 10: History example

If the machine leaves `process` while in `step_b`, and later transitions back to `process`, it will re-enter `step_b` directly (skipping `step_a`).

9 Orthogonal States (Parallelism)

An orthogonal state runs multiple child regions simultaneously.

Keyword: `orthogonal: true`

```
parallel_task:
  orthogonal: true
  states:
    region_a:
      initial: idle_a
      states:
        idle_a: {}
        working_a: {}
    region_b:
      initial: idle_b
      states:
        idle_b: {}
        working_b: {}
```

Listing 11: Orthogonal state definition

When `parallel_task` is entered, **all** child regions are entered simultaneously. Each region ticks independently. When exiting, all regions are exited.

The state path for orthogonal states uses bracket notation: `/parallel_task/[region_a/idle_a,region_b/]`

9.1 Safety: `transition_fired`

When a transition fires in one region, the other regions' ticks are skipped for that cycle to prevent inconsistent state. This is handled automatically via the `transition_fired` flag.

9.2 Fork Transitions (Drilling Down)

You can enter an orthogonal state by targeting its children directly.

1. **Implicit Fork:** to: `/run/g/a/sub_state`

The generator forces region a to `sub_state`. Other regions start at their `initial` states.

2. **Explicit Fork:** to: `/run/g/[a/sub_state, b/other]`

Explicitly sets the target for multiple regions at once.

9.3 Cross-Limb Transitions

A transition from one region to another region within the same orthogonal parent is a **cross-limb transition**. The generator handles this by exiting the target region's current state and entering the new target, while the source region remains unaffected.

9.4 Joining (Synchronization)

There are two ways to join parallel regions.

Manual join (all versions): Place a transition on the **parent** or on any state that checks `IN_STATE()` for all regions simultaneously.

```
main_task:
  orthogonal: true
  states:
    A:
      initial: working
      states:
```

```

    working:
      transitions:
        - guard: ctx.done_a
          to: finished
    finished: {}
  B:
    initial: working
    states:
      working:
        transitions:
          - guard: ctx.done_b
            to: finished
      finished: {}

# Transition on the parent container
transitions:
  - guard: IN_STATE(root_main_task_A_finished) && IN_STATE(
    root_main_task_B_finished)
    to: next_step

```

Listing 12: Manual join pattern (Rust guards)

Note: Use the `IN_STATE(X)` macro in guards for portable state checks. In Rust and Python, it expands to `ctx.in_state_X()`. In TypeScript, it expands to `ctx.inState_X()`. In C, it expands to the `IN_STATE_X` macro.

AND join nodes (`SM-builder-version: "0.6.0"+`): For a cleaner, declaration-based approach that composes across any number of sources and any hierarchy depth, use an `ands:` block (see Section 11). The compiler generates the compound `IN_STATE` guards automatically.

10 Decisions (Logic Trees)

Decisions allow you to externalize branching logic. A decision node can point to states, or **chain to other decisions**.

Restriction: You cannot use `action` or `hooks` inside a decision block. Actions must be placed on the transition *leading to* the decision, or on the state transitions following it.

10.1 Legacy Mode (`SM-builder-version: "0.6.0"`)

Decisions are collected into a **flat global namespace**; names must be unique across the entire model. All `@` references are bare (`@Name`) and look up from this global dict.

10.2 Hierarchical Mode (`SM-builder-version: "0.6.0" and later`)

Add `SM-builder-version: "0.6.0"` (or higher) at the top of your file to enable hierarchical pseudo-state scoping. Each `decisions:` block is owned by the state it is defined in, and `@` references are **path-qualified** — they resolve relative to the calling state's parent container, not a global dict.

Benefits:

- Multiple states can each define a `@Check` decision without name collisions.
- References are local by default but can cross container boundaries when needed.

References to pseudo-states follow the same path syntax as references to states (see Section 7.4): the prefix (`./`, `../`, `/absolute/`, bare sibling, etc.) navigates to a container, and `@Name` selects the pseudo-state within it instead of a child state.

Note: Target paths inside decision *rules* (the `to:` fields inside the decision's own rule list) resolve from the **decision's own scope**, not from the calling state. This is the same behaviour as in legacy mode.

10.3 Defining Decisions

Decisions can be defined at the root level or within any state.

```
decisions:
  validate_input:
    - guard: ctx.input < 0
      to: /error/negative
    - guard: ctx.input > 100
      to: "@check_permissions" # Chain to another decision
    - to: /process/calc # Default (no guard)

  check_permissions:
    - guard: ctx.is_admin
      to: /admin/override
    - to: /error/denied
```

Listing 13: Decision tree definition

10.4 Using Decisions

Reference a decision with the `@` prefix in the `to:` field:

```
transitions:
  - guard: ctx.data_ready
    to: "@validate_input"
```

Listing 14: Using a decision in a transition

Decisions can also be defined locally within a state and referenced with a bare name from sibling states:

```
my_state:
  decisions:
    local_check:
      - guard: ctx.x > 0
        to: positive
      - to: negative
  initial: waiting
  states:
    waiting:
      transitions:
        - to: "local_check"
    positive: {}
    negative: {}
```

Listing 15: Local decision definition

11 AND Join Nodes

AND join nodes (available in `SM-builder-version: "0.6.0"` and later) provide a declarative way to synchronize multiple concurrent activities. An AND node fires only when **all** of its source states are simultaneously active and all per-source guards are satisfied. This replaces the need to manually write `IN_STATE()` conjunctions or to chain decision nodes for synchronization.

11.1 Defining AND Nodes

Use the `ands:` block at any container level, with the same structure as `decisions:`.

```
SM-builder-version: "0.6.0"
language: python

initial: Parallel

states:
  Parallel:
    orthogonal: true
    ands:
      J1:
        # Name of the AND join node
        - to: /Done      # What to do when ALL sources are ready
    states:
      RegA:
        initial: Working
        states:
          Working:
            transitions:
              - guard: ctx.step >= 2
                to: "../../@J1"    # Reference J1 two levels up (inside
                                   Parallel)
      RegB:
        initial: Processing
        states:
          Processing:
            transitions:
              - guard: ctx.step >= 3
                to: "../../@J1"
    Done:
      transitions:
        - to: null
```

Listing 16: AND node definition inside an orthogonal state

The generated code for `Working`'s transition automatically becomes:

```
# Guard for Working -> @J1:
if ctx.step >= 2:
    if ctx.in_state_root_Parallel_RegB_Processing() and ctx.step >= 3:
        # — fire AND's outgoing rule: to /Done —
```

The compound condition (`in_state(Other)` and `other_guard`) is generated automatically from the reverse-edge information collected at compile time. No manual `IN_STATE` conjunctions needed.

11.2 Use Case 1 — Synchronization Join (Sibling Regions)

This is the most common pattern: two or more regions inside the same orthogonal state each independently work toward a “ready” sub-state, and the whole machine proceeds only when all regions are ready simultaneously.

```
SM-builder-version: "0.6.0"

Parallel:
  orthogonal: true
  ands:
    AllDone:
```

```

- to: /Final
states:
  RegA:
    initial: A
    states:
      A:
        transitions:
          - guard: ctx.step >= 2
            to: "../../@AllDone"
  RegB:
    initial: B
    states:
      B:
        transitions:
          - guard: ctx.step >= 3
            to: "../../@AllDone"
  RegC:
    initial: C
    states:
      C:
        transitions:
          - guard: ctx.step >= 4
            to: "../../@AllDone"

```

Listing 17: 3-way synchronization join

The join fires at **step** = 4 — when the last region (RegC) becomes ready. At **step** = 2 and **step** = 3, the composite guard blocks even though RegA or RegB are ready, because the other regions have not yet satisfied their guards.

Transition ordering is preserved: if a source state has higher-priority transitions listed *before* its AND edge, those transitions take precedence and the AND gate is suppressed for that cycle, matching the normal UML priority rule.

11.3 Use Case 2 — Cross-Boundary Join (Any Combination of Sources)

AND sources do not need to be in the same orthogonal state. They can come from any states anywhere in the tree — different nesting depths, different branches, or even non-orthogonal states.

```

SM-builder-version: "0.6.0"

ands:
  # Root-level AND node
  J1:
    - to: /Done

initial: Root
states:
  Root:
    orthogonal: true
    states:
      OrthoA:
        orthogonal: true # Nested orthogonal
        states:
          Worker:
            initial: ActiveA
            states:
              ActiveA:
                transitions:

```



```

        - guard: ctx.step >= 2
        to: "/@J1" # Absolute reference to root-level AND
OrthoB:
  initial: ActiveB
  states:
    ActiveB:
      transitions:
        - guard: ctx.step >= 3
        to: "/@J1"
Done:
  transitions:
    - to: null

```

Listing 18: AND node at root level, sources from nested composites

ActiveA lives at depth 5 inside a doubly-nested orthogonal region; ActiveB is at depth 4 in a sibling region. Both reference the same root-level join node via /@J1. The generated guard for ActiveA is:

```

if ctx.step >= 2:
    if ctx.in_state_root_Root_OrthoB_ActiveB() and ctx.step >= 3:
        # fire J1's rule: to /Done

```

11.4 Use Case 3 — Replacing Chained Decisions

Without AND nodes, synchronizing two flags often required chaining two decisions:

```

decisions:
  check_both:
    - guard: "IN_STATE(root_Parallel_RegA_Done) && IN_STATE(
      root_Parallel_RegB_Done)"
    to: /Final
    - to: stay_here # dummy self-loop

```

Listing 19: Old pattern: manual chaining

With an AND node, the same semantics are expressed declaratively:

```

ands:
  BothDone:
    - to: /Final

# Then in each region's Done state:
transitions:
  - to: "@BothDone"

```

Listing 20: New pattern: AND node (0.6.0+)

The AND node is less error-prone (no manual state name strings), adapts automatically when states are renamed, and composes cleanly across N sources without O(N) chaining.

11.5 Per-Source Guards and the time Variable

Each incoming edge to an AND node may carry its own guard. When the AND fires from source state X, the generated code ANDs in every other source's `in_state()` check *plus* that source's guard expression.

If a guard uses the special `time` variable, it is automatically rebased to that source state's own entry timestamp, not the firing state's timer:

```

# Guard in RegB: time >= 2.0 -> fires if RegB has been active >= 2 s
# When RegA fires the join, the generated check for RegB is:
if ctx.in_state_RegB() and (ctx.now - ctx.state_timers[regb_id]) >= 2.0:
    ...

```

11.6 AND Node Rules

AND outgoing rules follow exactly the same syntax as decision rules: a list of {guard, to} entries evaluated top-to-bottom. The first matching rule fires. Rules inside the AND resolve paths from the AND node's own scope (the container it is defined in).

12 Complete Example

Below is a minimal but complete SMB file demonstrating the key features, with Rust as the target language.

```

language: rust

includes: |
    fn log(msg: &str) { println!("{}", msg); }

context: |
    pub counter: i32,

context_init: |
    counter: 0,

hooks:
    entry: |
        println!("Entering_{}", state_full_name);
    exit: |
        println!("Exiting_{}", state_full_name);

initial: active

states:
    active:
        initial: idle
        states:
            idle:
                transitions:
                    - guard: ctx.counter >= 3
                      to: working
            working:
                do: |
                    ctx.counter += 1;
                transitions:
                    - guard: ctx.counter >= 10
                      to: null # Terminate

error:
    entry: |
        log("ERROR_STATE");

```

Listing 21: Complete SMB example (Rust)